OMI-Lisp Portable Runtime Engine

Normative User-Facing Implementation Specification

Version: 0.1.0
Status: Canonical Implementation Draft
Primary implementation language: ISO C11
Reference implementation: `omi_lisp.py`
Primary artifact: embeddable `libomi` runtime
Authority boundary: deterministic interpretation runtime; validation and receipt determine acceptance

---

0. Normative Language

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHOULD**, **SHOULD NOT**, and **MAY** are normative.

```
MUST
required for conforming behavior

SHOULD
recommended unless a documented profile gives a reason otherwise

MAY
optional behavior

implementation-defined
must be documented by the implementation

profile-defined
must be selected by a named OMI runtime profile
```

A runtime is conforming only if:

```
its parser behavior is deterministic

its value representation preserves language semantics

its canonical serialization preserves CAR/CDR order

its receipt inputs are explicit

its configured quasigroup operations are reversible
```

```
its conformance vectors pass
```

---

1. Purpose

OMI-Lisp is a small deterministic Lisp-family runtime designed to provide:

```
portable symbolic evaluation

ordered dotted CONS relations

bounded bitboard computation

QuQuart interpretation registers

tetrahedral centroid reduction

quasigroup and Latin-square resolution

scope-aware OMI notation

receipt replay stability

foreign-language embedding
```

The implementation goal is:

```
one semantic C core
with many host-language adapters
```

The normative runtime artifact is:

```
libomi
```

Optional artifacts include:

```
omi
command-line interpreter

omi-repl
```

```
interactive shell

omi-wasm
WebAssembly module

language bindings
Python, JavaScript, Rust, Go, Java, and others
```

---

2. Non-Goals

The base runtime is not:

```
a quantum computer

a database management system

an operating system

a networking protocol

a cryptographic protocol by itself

a global semantic authority

an automatic application executor
```

OMI-Lisp MAY be embedded within such systems.

The runtime itself stops at:

```
parse

evaluate

resolve

validate

receipt

project
```

Application-specific effects remain outside the core unless explicitly supplied through a host capability.

---

3. Core Doctrine

The runtime follows this interpretation sequence:

```
Q0 source
→
Q1 notation
→
Q2 reading
→
Q3 receipt
```

The four QuQuart states are:

```
Q0
source

Q1
notation

Q2
reading

Q3
receipt
```

The associated `.o` roles are:

```
RULES.o
source law

FACTS.o
grounded source

COMBINATORS.o
reading operations

CLOSURES.o
closure and receipt boundary
```

```
CONS.o
centroid reducer and reversible resolver
```

Canonical mapping:

```
Q0 = RULES.o + FACTS.o

Q1 = declared or generated notation

Q2 = COMBINATORS.o(Q0,Q1)

Q3 = CLOSURES.o(Q0,Q1,Q2)

CONS.o = centroid reduction over Q0-Q3
```

`CONS` is not `Q4`.

---

4. Runtime Architecture

A conforming implementation SHOULD be separated into these layers:

```
libomi-core
parser, evaluator, values, CONS, symbols,
canonical serialization, QuQuart, quasigroup,
validation, and receipt construction

libomi-host
allocation, storage, logging, module loading,
monotonic counters, clocks, and optional capabilities

omi-cli
file execution, command-line evaluation, JSON output,
receipt-ring persistence, and REPL
```

The core MUST NOT require:

```
a filesystem

threads

sockets
```

```
environment variables

a terminal

dynamic linking

floating-point arithmetic

JSON
```

These MAY be provided through profiles or host adapters.

---

5. Source Organization

Recommended repository layout:

```
include/
  omi/
    omi.h
    config.h
    status.h
    value.h
    runtime.h
    host.h
    parser.h
    evaluator.h
    environment.h
    symbol.h
    cons.h
    serialize.h
    hash.h
    scope.h
    governor.h
    ququart.h
    centroid.h
    quasigroup.h
    validation.h
    receipt.h
    ring.h
    module.h

src/
  runtime.c
  value.c
```

```
  parser.c
  lexer.c
  evaluator.c
  environment.c
  symbol.c
  cons.c
  serialize.c
  hash.c
  scope.c
  governor.c
  ququart.c
  centroid.c
  quasigroup.c
  validation.c
  receipt.c
  ring.c
  builtins.c
  delta16.c
  bqf.c

cli/
  main.c
  repl.c
  json.c
  file_host.c

ports/
  python/
  node/
  wasm/
  rust/
  go/
  java/

profiles/
  minimal/
  hosted/
  embedded/
  wasm/
  deterministic/

tests/
  lexer_test.c
  parser_test.c
  value_test.c
  cons_test.c
  evaluator_test.c
  environment_test.c
```

```
  scope_test.c
  governor_test.c
  quasigroup_test.c
  ququart_test.c
  centroid_test.c
  receipt_test.c
  ring_test.c
  serialization_test.c
  conformance_test.c

vectors/
  syntax/
  evaluation/
  quasigroup/
  ququart/
  receipt/
  serialization/
```

---

6. Public C ABI

The public ABI MUST use opaque runtime objects and stable value handles.

```c
#ifndef OMI_H
#define OMI_H

#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>

#ifdef __cplusplus
extern "C" {
#endif

typedef struct omi_runtime omi_runtime;
typedef uint64_t omi_handle;

typedef enum omi_status {
    OMI_OK = 0,

    OMI_ERR_ARGUMENT,
    OMI_ERR_CONFIG,
    OMI_ERR_MEMORY,
    OMI_ERR_LIMIT,
```

```
    OMI_ERR_TOKEN,
    OMI_ERR_PARSE,
    OMI_ERR_EOF,
    OMI_ERR_DOT,
    OMI_ERR_TRAILING_INPUT,

    OMI_ERR_UNBOUND,
    OMI_ERR_TYPE,
    OMI_ERR_ARITY,
    OMI_ERR_NOT_CALLABLE,
    OMI_ERR_EVALUATION,

    OMI_ERR_PROFILE,
    OMI_ERR_QUASIGROUP,
    OMI_ERR_NOT_REVERSIBLE,
    OMI_ERR_VALIDATION,
    OMI_ERR_RECEIPT,
    OMI_ERR_STORAGE,
    OMI_ERR_UNSUPPORTED
} omi_status;

typedef struct omi_slice {
    const uint8_t *data;
    size_t size;
} omi_slice;

typedef struct omi_mut_slice {
    uint8_t *data;
    size_t size;
} omi_mut_slice;

typedef struct omi_error {
    omi_status status;
    size_t byte_offset;
    size_t line;
    size_t column;
    char message[192];
} omi_error;

omi_runtime *omi_runtime_create(
    const struct omi_config *config,
    const struct omi_host *host,
    omi_error *error
);

void omi_runtime_destroy(omi_runtime *runtime);

omi_status omi_parse_utf8(
```

```
    omi_runtime *runtime,
    const char *source,
    size_t source_length,
    omi_handle *out_form,
    omi_error *error
);

omi_status omi_eval(
    omi_runtime *runtime,
    omi_handle form,
    omi_handle *out_value,
    omi_error *error
);

omi_status omi_eval_utf8(
    omi_runtime *runtime,
    const char *source,
    size_t source_length,
    omi_handle *out_value,
    omi_error *error
);

omi_status omi_format(
    omi_runtime *runtime,
    omi_handle value,
    char *buffer,
    size_t capacity,
    size_t *required,
    omi_error *error
);

omi_status omi_serialize_canonical(
    omi_runtime *runtime,
    omi_handle value,
    uint8_t *buffer,
    size_t capacity,
    size_t *required,
    omi_error *error
);

#ifdef __cplusplus
}
#endif

#endif
```

The public ABI MUST NOT expose:

```
internal pointers

arena addresses

garbage-collector metadata

host-language object pointers

internal symbol-table buckets
```

---

7. Runtime Configuration

```c
typedef enum omi_numeric_profile {
    OMI_NUMERIC_SYMBOLIC = 0,
    OMI_NUMERIC_FIXED,
    OMI_NUMERIC_FLOAT,
    OMI_NUMERIC_SOFT
} omi_numeric_profile;

typedef enum omi_memory_profile {
    OMI_MEMORY_FIXED = 0,
    OMI_MEMORY_HOSTED,
    OMI_MEMORY_CUSTOM
} omi_memory_profile;

typedef struct omi_limits {
    uint32_t max_cons_cells;
    uint32_t max_symbols;
    uint32_t max_symbol_bytes;
    uint32_t max_parse_depth;
    uint32_t max_eval_depth;
    uint32_t max_call_arguments;
    uint32_t max_lambda_parameters;
    uint32_t max_receipts;
    uint32_t max_modules;
    uint32_t max_canonical_bytes;
} omi_limits;

typedef struct omi_config {
    uint32_t abi_version;

    omi_numeric_profile numeric_profile;
    omi_memory_profile memory_profile;
```

```
    omi_limits limits;

    bool enable_lambda;
    bool enable_define;
    bool enable_receipts;
    bool enable_receipt_ring;
    bool enable_generalized_mean;
    bool enable_quasigroup;
    bool enable_centroid;
    bool enable_host_functions;

    uint16_t default_quasigroup_profile;
    uint16_t default_validation_profile;
    uint16_t default_receipt_profile;

    void *fixed_memory;
    size_t fixed_memory_size;
} omi_config;
```

All limits MUST be enforced.

Failure to satisfy a configured limit MUST return `OMI_ERR_LIMIT`; it MUST NOT silently truncate structural data.

---

8. Host Interface

```
typedef enum omi_log_level {
    OMI_LOG_DEBUG = 0,
    OMI_LOG_INFO,
    OMI_LOG_WARN,
    OMI_LOG_ERROR
} omi_log_level;

typedef struct omi_host {
    void *context;

    void *(*allocate)(
        void *context,
        size_t size,
        size_t alignment
    );

    void (*deallocate)(
```

```
        void *context,
        void *pointer,
        size_t size,
        size_t alignment
    );

    omi_status (*load_module)(
        void *context,
        omi_slice module_name,
        omi_mut_slice *output
    );

    omi_status (*store_receipt)(
        void *context,
        omi_slice canonical_receipt
    );

    omi_status (*load_receipt)(
        void *context,
        uint64_t receipt_id,
        omi_mut_slice *output
    );

    uint64_t (*monotonic_counter)(void *context);

    void (*log)(
        void *context,
        omi_log_level level,
        omi_slice message
    );
} omi_host;
```

Host callbacks MUST be optional unless required by the selected profile.

The core MUST return `OMI_ERR_UNSUPPORTED` when an unavailable host capability is requested.

---

9. Value Model

The runtime MUST support:

```
NIL

boolean
```

```
integer

optional real number

symbol

string

CONS pair

native function

lambda closure

QuQuart value

centroid result

gauge result

receipt

external host handle
```

A recommended value representation is a tagged 64-bit handle:

```
typedef uint64_t omi_value;

typedef enum omi_tag {
    OMI_TAG_SPECIAL = 0,
    OMI_TAG_INT     = 1,
    OMI_TAG_SYMBOL  = 2,
    OMI_TAG_CONS    = 3,
    OMI_TAG_NATIVE  = 4,
    OMI_TAG_OBJECT  = 5,
    OMI_TAG_EXTERN  = 6,
    OMI_TAG_RESERVED = 7
} omi_tag;
```

Low bits MAY encode tags.

The exact bit layout is implementation-defined, but:

```
serialized values MUST NOT depend on pointer values
```

```
value equality MUST follow language semantics

integer overflow behavior MUST be documented

external handles MUST never be accepted as canonical source authority
```

---

10. Memory Model

The runtime SHOULD use arena-backed handles.

```c
typedef struct omi_cons_cell {
    omi_value car;
    omi_value cdr;
} omi_cons_cell;

typedef struct omi_cons_arena {
    omi_cons_cell *cells;
    uint32_t capacity;
    uint32_t used;
    uint32_t free_head;
} omi_cons_arena;
```

A CONS handle MUST refer to a valid arena entry.

The runtime MUST detect:

```
invalid handles

out-of-range handles

freed handles, when individual freeing exists

allocation exhaustion

recursive cycles during canonical serialization
```

The base language MAY permit cyclic runtime pairs, but canonical serialization MUST reject cycles unless a named graph-serialization profile is selected.

---

11. Lexical Grammar

The base lexical grammar includes:

```
left parenthesis
(

right parenthesis
)

dot
.

quote prefix
'

booleans
#t #f

hexadecimal integers
0x...

decimal integers

optional floating values

symbols

strings

comments beginning with ;
```

Whitespace separates tokens.

Comments continue to the next newline.

A conforming tokenizer MUST recognize strings with escaped characters.

Base token grammar:

```
program      = { form } ;

form         = atom
             | list
```

```
              | quote ;

quote         = "'" form ;

list          = "(" [ list_body ] ")" ;

list_body     = form { form }
              | form { form } "." form ;

atom          = boolean
              | hexadecimal
              | integer
              | real
              | string
              | symbol ;

boolean       = "#t" | "#f" ;

hexadecimal   = "0x" hex_digit { hex_digit } ;

integer       = [ "+" | "-" ] digit { digit } ;

symbol        = symbol_initial { symbol_continue } ;
```

A dot outside a list MUST produce `OMI_ERR_DOT`.

A dotted list MUST have at least one CAR item and exactly one final CDR.

---

12. Pair and List Semantics

A pair is:

```
(car . cdr)
```

A proper list:

```
(A B C)
```

is semantically:

```
(A . (B . (C . NIL)))
```

An improper list:

```
(A B . C)
```

is semantically:

```
(A . (B . C))
```

The runtime MUST preserve the distinction.

Functions requiring a proper list MUST reject improper lists.

`car` and `cdr` MUST reject non-pair values.

---

13. Core Evaluation Semantics

Self-evaluating values:

```
NIL
booleans
integers
real values
strings
native objects that declare self-evaluation
```

Symbols are resolved through lexical environments.

Pair evaluation treats the CAR as an operator position unless a special form intercepts evaluation.

Required special forms:

```
quote
if
begin
define
```

```
lambda
cite
```

Required procedures:

```
cons
car
cdr
null?
pair?
list

+
-
*
/
=
<
>
not
```

Profile-defined procedures MAY include:

```
xor
rotl16
rotr16
delta16
bqf32
hash
normalize-notation
compile-delta
gauge-resolve
validate
receipt
ring-count
quasigroup-resolve
quasigroup-left-divide
quasigroup-right-divide
ququart
centroid-reduce
```

---

14. Special Forms

14.1 `quote`

```
(quote value)
'value
```

Returns `value` without evaluation.

Arity MUST be exactly one.

14.2 `if`

```
(if condition consequent)
(if condition consequent alternate)
```

Only the selected branch is evaluated.

False values are:

```
#f
NIL
```

All other values are true.

14.3 `begin`

```
(begin form...)
```

Evaluates forms left-to-right and returns the last value.

An empty `begin` returns `NIL`.

14.4 `define`

```
(define name expression)
```

Evaluates the expression and binds it in the current environment.

`name` MUST be a symbol.

14.5 `lambda`

```
(lambda (parameter...) body...)
```

Creates a lexical closure.

The parameter list MUST be proper.

Arity MUST match exactly unless a later variadic profile is declared.

14.6 `cite`

```
(cite expression)
```

Evaluates `expression`, validates it under the active profile, constructs a receipt candidate, and returns a receipt object.

`cite` MUST NOT treat parsing success as acceptance.

---

15. Environment Semantics

Environments form a parent-linked lexical chain.

```
typedef struct omi_binding {
    omi_symbol_id symbol;
    omi_value value;
} omi_binding;

typedef struct omi_environment {
    struct omi_environment *parent;
    omi_binding *bindings;
    uint32_t count;
    uint32_t capacity;
} omi_environment;
```

Lookup MUST search:

```
current environment

then parent environments in order
```

An unresolved symbol MUST produce `OMI_ERR_UNBOUND`.

Closures MUST retain access to their lexical parent environment.

---

16. OMI Scope Model

Canonical scopes:

```
FS
GS
RS
US
```

Canonical masks:

```
FS = 0x0001
GS = 0x0010
RS = 0x0100
US = 0x1000
```

Subpath masks:

```
o = 0x000F
/ = 0x00F0
? = 0x0F00
@ = 0xF000
```

A runtime MUST preserve scope names separately from numeric masks.

A scope object SHOULD contain:

```
typedef enum omi_scope {
    OMI_SCOPE_FS = 0,
    OMI_SCOPE_GS,
    OMI_SCOPE_RS,
```

```
    OMI_SCOPE_US
} omi_scope;

typedef struct omi_scope_value {
    omi_scope scope;
    uint16_t mask;
} omi_scope_value;
```

---

17. Omicron Readiness and Earned Dot

The strict OMI declaration profile MAY require dotted executable declarations to be unavailable until the Omicron state reaches readable space.

Canonical transition:

```
pre-header
→
pre-language topology
→
SP crossing
→
dot earned
→
dotted execution enabled
```

A runtime implementing this profile MUST maintain:

```
typedef enum omi_stage {
    OMI_STAGE_INITIAL = 0,
    OMI_STAGE_PREHEADER,
    OMI_STAGE_PRELANGUAGE,
    OMI_STAGE_READABLE
} omi_stage;

typedef struct omi_omicron_state {
    omi_stage stage;
    bool dot_earned;
    uint8_t active_plane;
    uint8_t active_byte;
} omi_omicron_state;
```

Attempting to evaluate a dotted executable declaration before `dot_earned` MUST fail with a dedicated error.

Plain Lisp data parsing MAY remain available if the host selects a relaxed parser profile.

---

18. Five Governors

Canonical governors:

```
FACTS       p = -1
RULES       p = 0
CLOSURES    p = 1
COMBINATORS p = 2
CONS        p = 3
```

Their operational roles are:

```
FACTS
inverse or grounded constraint

RULES
Genesis/equality pivot

CLOSURES
ordered completion

COMBINATORS
quadratic composition

CONS
centroid reduction and runtime embodiment
```

The exponent MUST be stored as metadata.

The generalized mean is an optional numeric interpretation of the governor, not the only meaning of the governor.

---

19. Generalized Mean Resolution

When enabled:

$$M_p(x_1, \ldots, x_n) = \left(\frac{1}{n} \sum_{i=1}^{n} x_i^p \right)^{1/p}$$

For $p = 0$:

$$M_0(x_1, \ldots, x_n) = \exp\left(\frac{1}{n} \sum_{i} \ln x_i \right)$$

The reference behavior requires positive inputs.

A portable implementation MUST select one numeric profile:

```
symbolic

fixed-point

host floating point

software-reproducible numeric
```

The symbolic profile records the operation without calculating an approximate floating result.

No receipt requiring cross-platform byte equality SHOULD depend on ordinary host floating-point unless its profile defines rounding and serialization exactly.

---

20. Required Arithmetic Primitives

20.1 Rotation

```c
uint16_t omi_rotl16(uint16_t x, unsigned count);
uint16_t omi_rotr16(uint16_t x, unsigned count);
```

Counts are reduced modulo 16.

20.2 Delta16

$$\Delta_{16}(x, c) = ROL_{16}(x, 1) \oplus ROL_{16}(x, 3) \oplus ROR_{16}(x, 2) \oplus c$$

```c
uint16_t omi_delta16(uint16_t x, uint16_t carry);
```

20.3 Binary Quadratic Fold

$$Q(x, y) = 60x^2 + 16xy + 4y^2$$

```
uint32_t omi_bqf32(uint16_t x, uint16_t y);
```

The implementation MUST define whether intermediate values are widened or modular.

The recommended behavior widens to at least 64 bits internally and returns the low 32 bits only when the result fits the declared profile.

---

21. QuQuart Register

```
typedef enum omi_ququart_state {
    OMI_Q0_SOURCE = 0,
    OMI_Q1_NOTATION,
    OMI_Q2_READING,
    OMI_Q3_RECEIPT
} omi_ququart_state;

typedef struct omi_ququart {
    omi_value source;
    omi_value notation;
    omi_value reading;
    omi_value receipt;

    uint8_t present_mask;
} omi_ququart;
```

Bit assignments:

```
bit 0
Q0 present

bit 1
Q1 present

bit 2
Q2 present
```

```
bit 3
Q3 present
```

The runtime MUST NOT create an implied Q4 for CONS.

---

22. Tetrahedral Centroid Model

Vertices:

```
RULES
FACTS
COMBINATORS
CLOSURES
```

Centroid:

```
CONS
```

```c
typedef struct omi_tetrahedral_vertices {
    omi_value rules;
    omi_value facts;
    omi_value combinators;
    omi_value closures;
} omi_tetrahedral_vertices;

typedef struct omi_centroid_result {
    omi_ququart ququart;

    uint8_t selector_mask;
    uint8_t edge_mask;
    uint8_t face_mask;

    omi_value result;
    uint64_t contrast_witness;
    uint8_t structural_digest[32];

    bool structurally_valid;
    bool accepted;
} omi_centroid_result;
```

---

23. Tetrahedral Edges

Canonical edge bits:

```
bit 0
RULES–FACTS

bit 1
RULES–COMBINATORS

bit 2
RULES–CLOSURES

bit 3
FACTS–COMBINATORS

bit 4
FACTS–CLOSURES

bit 5
COMBINATORS–CLOSURES
```

Each edge is evaluated by a profile-defined predicate.

A runtime MUST record which predicates were evaluated separately from whether they passed.

Recommended representation:

```c
typedef struct omi_predicate_mask {
    uint8_t evaluated;
    uint8_t passed;
} omi_predicate_mask;
```

This prevents:

```
not evaluated
```

from being confused with:

```
evaluated and failed
```

---

24. Tetrahedral Faces

Canonical face bits:

```
bit 0
RULES / FACTS / COMBINATORS

bit 1
RULES / FACTS / CLOSURES

bit 2
RULES / COMBINATORS / CLOSURES

bit 3
FACTS / COMBINATORS / CLOSURES
```

Face predicates are profile-defined.

The default strict centroid policy SHOULD require all declared required faces to pass.

---

25. Boolean Continuation Ladder

For selector rank n:

$$|CAR_n| = 2^n$$

One Boolean CDR truth table contains:

$$2^n \text{ output bits}$$

The number of possible CDR functions is:

$$2^{2^n}$$

Normative table:

Rank	CAR assignments	One CDR width	Possible CDR functions
0	1	1 bit	2
1	2	2 bits	4
2	4	4 bits	16

Rank	CAR assignments	One CDR width	Possible CDR functions
3	8	8 bits	256
4	16	16 bits	2^{16}
5	32	32 bits	2^{32}
6	64	64 bits	2^{64}
7	128	128 bits	2^{128}
8	256	256 bits	2^{256}

The implementation MUST distinguish:

```
rank

assignment count

selected truth-table width

population of possible truth tables
```

---

26. Rank-8 Predicate Representation

```c
typedef struct omi_word256 {
    uint64_t lane[4];
} omi_word256;
```

Bit lookup:

```c
static inline bool omi_word256_test(
    const omi_word256 *word,
    uint8_t index
) {
    uint8_t lane = index >> 6;
    uint8_t bit = index & 63u;

    return ((word->lane[lane] >> bit) & 1u) != 0u;
}
```

A selected rank-8 CDR occupies exactly 256 bits.

The runtime MUST NOT attempt to enumerate the 2^{256} possible predicates.

---

27. Quasigroup Resolver

A quasigroup profile MUST implement:

```
multiply
CAR × CDR → CONS

left divide
CAR × CONS → CDR

right divide
CONS × CDR → CAR
```

```c
typedef struct omi_quasigroup_profile {
    uint16_t profile_id;
    uint16_t width_bits;

    const char *name;

    omi_status (*multiply)(
        void *context,
        const uint8_t *car,
        const uint8_t *cdr,
        uint8_t *cons,
        size_t width_bytes
    );

    omi_status (*left_divide)(
        void *context,
        const uint8_t *car,
        const uint8_t *cons,
        uint8_t *cdr,
        size_t width_bytes
    );

    omi_status (*right_divide)(
        void *context,
        const uint8_t *cons,
        const uint8_t *cdr,
        uint8_t *car,
        size_t width_bytes
```

```
    );

    void *context;
} omi_quasigroup_profile;
```

A conforming profile MUST satisfy for all test vectors:

```
left_divide(car, multiply(car,cdr)) = cdr

right_divide(multiply(car,cdr),cdr) = car
```

---

28. Affine Bitboard Quasigroup

A recommended family is:

$$a * b = P(a) \oplus Q(b) \oplus k$$

where:

```
P
invertible CAR permutation

Q
invertible CDR permutation

k
profile constant
```

Recovery:

$$b = Q^{-1}(c \oplus P(a) \oplus k)$$

$$a = P^{-1}(c \oplus Q(b) \oplus k)$$

CAR and CDR MUST use distinct transforms where ordered interpretation is required.

Plain XOR MUST NOT be the canonical structural resolver.

---

29. Latin-Square Conformance

For small finite profiles, the implementation MUST verify:

```
every symbol appears once per row

every symbol appears once per column

every CAR/CDR pair resolves uniquely

left and right recovery are unique
```

For large algorithmic profiles, exhaustive enumeration MAY be replaced by:

```
formal construction proof

property-based testing

declared invertible permutations

published test vectors
```

---

30. F* Resolver Selection

```c
typedef struct omi_resolver_selector {
    uint8_t family;
    uint8_t dialect;

    uint16_t profile_id;

    uint16_t car_permutation_id;
    uint16_t cdr_permutation_id;
    uint16_t symbol_permutation_id;
} omi_resolver_selector;
```

Canonical interpretation:

```
F
resolver family

*
isotopic dialect
```

Changing the dialect MUST NOT silently change the field width or receipt profile.

Such changes require explicit versioned metadata.

---

31. Polyadic Centroid Resolver

A tetrahedral centroid MAY use a four-input polyadic quasigroup:

$$CONS = F(RULES, FACTS, COMBINATORS, CLOSURES)$$

The profile MUST define recovery functions for each coordinate:

```
recover RULES

recover FACTS

recover COMBINATORS

recover CLOSURES

recover CONS
```

```
typedef struct omi_polyadic_profile {
    uint16_t profile_id;
    uint16_t width_bits;

    omi_status (*resolve)(
        void *context,
        const uint8_t inputs[4][32],
        uint8_t output[32]
    );

    omi_status (*recover)(
        void *context,
        unsigned missing_index,
        const uint8_t known[4][32],
        uint8_t recovered[32]
    );

    void *context;
} omi_polyadic_profile;
```

The exact fixed dimensions MAY be generalized in production.

---

32. Native Function Registry

```
typedef omi_status (*omi_native_fn)(
    omi_runtime *runtime,
    const omi_value *arguments,
    size_t argument_count,
    omi_value *out,
    omi_error *error
);

typedef struct omi_native_entry {
    const char *name;
    omi_native_fn function;

    uint16_t minimum_arity;
    uint16_t maximum_arity;

    uint32_t capability_flags;
} omi_native_entry;
```

The runtime MUST validate arity before invoking the function.

Native functions MUST NOT retain raw pointers to movable runtime objects unless the selected memory profile guarantees stability.

---

33. Capability Isolation

Native procedures SHOULD declare required capabilities:

```
pure

allocation

receipt

module loading

external handle

clock

storage
```

```
logging

unsafe host call
```

A runtime MUST deny calls requiring capabilities absent from its profile.

User-facing code MUST NOT gain ambient filesystem, networking, or process access merely by evaluating an OMI-Lisp form.

---

34. Validation

Parsing and evaluation are not validation.

Validation is profile-driven.

```
typedef struct omi_validation_context {
    omi_value source;
    omi_value notation;
    omi_value reading;
    omi_value result;

    const omi_centroid_result *centroid;

    uint16_t profile_id;
} omi_validation_context;

typedef struct omi_validation_result {
    bool accepted;
    uint32_t reason_code;
    char reason[160];
} omi_validation_result;
```

Validation SHOULD consider:

```
type correctness

scope correctness

required edge predicates

required face predicates
```

```
quasigroup reversibility

canonical serialization success

receipt ancestry

host monotonic counter

profile-specific closure rules
```

No projection witness may independently accept state.

---

35. Receipt Model

```c
typedef struct omi_receipt {
    uint16_t version;
    uint16_t profile_id;

    uint16_t slot;
    uint8_t accepted;
    uint8_t reserved;

    uint64_t cycle;
    uint64_t monotonic_counter;

    uint8_t source_digest[32];
    uint8_t notation_digest[32];
    uint8_t reading_digest[32];
    uint8_t result_digest[32];
    uint8_t route_digest[32];
    uint8_t receipt_digest[32];

    uint16_t quasigroup_profile;
    uint16_t validation_profile;

    uint8_t ququart_mask;
    uint8_t edge_evaluated;
    uint8_t edge_passed;
    uint8_t face_evaluated;
    uint8_t face_passed;

    uint32_t reason_code;
} omi_receipt;
```

The actual digest width MAY vary by receipt profile.

A production receipt SHOULD use a cryptographic hash.

A lightweight non-cryptographic hash MUST be labeled as such.

---

36. Receipt Replay Stability

The normative law is:

```
same canonical source
same notation
same reading route
same resolver profile
same validation profile
same result
same receipt-parent context
→ same receipt identity
```

This is **Receipt Replay Stability**.

It does not claim that every operator satisfies mathematical idempotence.

A conformance test MUST evaluate the same input twice in fresh equivalent runtime states and compare canonical receipt identity.

---

37. Receipt Ring

Default ring size:

```
5040
```

Default address:

$$slot = fano7 \cdot 720 + role3 \cdot 240 + local240$$

Ranges:

```
fano7
0..6
```

```
role3
0..2

local240
0..239

slot
0..5039
```

```c
static inline uint16_t omi_slot5040(
    uint8_t fano7,
    uint8_t role3,
    uint8_t local240
) {
    return (uint16_t)(
        (uint16_t)fano7 * 720u +
        (uint16_t)role3 * 240u +
        local240
    );
}
```

Bounds MUST be checked before packing.

A receipt ring MUST NOT overwrite an existing accepted receipt silently.

The overwrite policy MUST be explicit:

```
reject

append elsewhere

retain history chain

replace only with matching identity

host-defined monotonic policy
```

---

38. Canonical Serialization

Canonical serialization MUST be:

```
ordered

typed

length-delimited

versioned

independent of memory addresses

independent of hash-table iteration order
```

Recommended type tags:

```
0x00 NIL

0x01 false

0x02 true

0x10 signed integer

0x11 unsigned integer

0x12 fixed-point

0x13 floating profile value

0x20 symbol

0x21 string

0x30 CONS

0x40 QuQuart

0x41 centroid

0x50 gauge result

0x60 receipt

0x70 external reference
```

Recommended CONS encoding:

```
CONS_TAG
VERSION
PROFILE_ID
CAR_LENGTH
CAR_CANONICAL_BYTES
CDR_LENGTH
CDR_CANONICAL_BYTES
```

CAR bytes MUST appear before CDR bytes.

---

39. Structural Identity and Contrast Witness

The runtime SHOULD expose both:

```
contrast witness

structural digest
```

Contrast witness:

```
cheap local comparison

may use XOR

not unique

not a receipt identity
```

Structural digest:

```
hash of ordered canonical serialization

binds type and structure

used by receipts
```

The pair:

```
(0xAA55 . 0x55AA)
```

MAY produce contrast witness:

```
0xFFFF
```

The reverse pair MUST have a different structural digest.

---

40. Error Model

Every failing public API MUST return:

```
status code

human-readable message

source location when available
```

Errors MUST NOT be communicated only through global variables.

Parser errors SHOULD include:

```
byte offset

line

column

unexpected token

expected construct
```

User-facing CLI errors SHOULD avoid exposing raw host addresses or internal allocator state.

---

41. REPL Behavior

The REPL SHOULD:

```
print one prompt

parse one or more complete forms
```

```
support multiline forms

print formatted results

print errors to stderr

continue after recoverable errors

exit cleanly on EOF
```

Default prompt:

```
omi>
```

The REPL MUST NOT automatically accept or receipt every expression.

Receipt creation occurs only through:

```
cite

an explicit host policy

a declared top-level receipt mode
```

---

42. CLI Behavior

Recommended interface:

```
omi [options] [file]

-e, --eval EXPR
evaluate one expression

--json
print JSON-compatible projection

--canonical
print canonical byte encoding as hexadecimal

--ring PATH
```

```
load and save receipt ring

--profile NAME
select runtime profile

--no-receipts
disable receipts

--strict-omicron
require earned dot notation

--version
print runtime and ABI versions
```

Exit codes SHOULD distinguish:

```
0 success

1 general runtime error

2 parse error

3 evaluation error

4 validation rejection

5 configuration error

6 host/storage error
```

---

43. JSON Projection

JSON is a projection, not the canonical binary encoding.

Recommended conversion:

```
NIL
null

symbol
string containing symbol spelling, with explicit type wrapper when ambiguity
matters
```

```
proper list
JSON array

improper pair
{"car": ..., "cdr": ...}

receipt
object with explicit fields

QuQuart
object with q0/q1/q2/q3

centroid
object with vertices, masks, result, and validation
```

Round-trip fidelity is not guaranteed through plain JSON unless the typed JSON profile is used.

---

44. Module System

The minimal module contract SHOULD load:

```
RULES.o

FACTS.o

COMBINATORS.o

CLOSURES.o

CONS.o
```

Each module MUST declare:

```
module type

version

canonical profile

dependencies

exports
```

```
required host capabilities

canonical digest
```

The host resolves module bytes.

The runtime parses and validates them.

Loading a module MUST NOT automatically execute arbitrary host effects.

---

45. Module Roles

`RULES.o`

Contains:

```
source laws

type constraints

allowed operators

resolver declarations

validation requirements
```

`FACTS.o`

Contains:

```
grounded symbols

constants

source declarations

bounded observations
```

`COMBINATORS.o`

Contains:

```
pure functions

native operation declarations

quasigroup profiles

projection functions
```

`CLOSURES.o`

Contains:

```
edge predicates

face predicates

receipt policy

replay policy

acceptance constraints
```

`CONS.o`

Contains:

```
centroid reducer

CAR/CDR resolver profile

polyadic recovery profile

selector masks

canonical structural declarations
```

---

46. Portability Profiles

46.1 Minimal

```
fixed arena

integers only

no lambda

no filesystem

no receipts

ordered pairs

parser and evaluator

quasigroup optional
```

46.2 Embedded

```
fixed arena

bounded symbols

no host floating point

static native registry

optional receipt callback

no dynamic modules

deterministic execution limits
```

46.3 Hosted

```
dynamic allocation

lambda and define
```

```
file loading

receipt-ring persistence

JSON projection

interactive REPL
```

46.4 WASM

```
linear-memory arena

opaque integer handles

host imports

no direct file access

canonical bytes at ABI boundary
```

46.5 Deterministic

```
no host floating point

fixed hash profile

fixed symbol normalization

fixed module order

fixed serialization

fixed receipt inputs
```

---

47. Language Bindings

All bindings MUST call the C ABI rather than independently redefining semantics, unless they are explicitly labeled reference or experimental implementations.

Python

Recommended options:

```
cffi

CPython limited API extension

ctypes for initial testing
```

Node.js

Recommended:

```
N-API native addon

or WASM
```

Rust

```
extern "C"
```

with a safe RAII wrapper.

Go

Use cgo with serialized byte boundaries.

Java

Use JNI or Foreign Function and Memory API.

Bindings MUST preserve:

```
handle lifetime

error details

canonical serialization bytes

receipt bytes
```

```
CAR/CDR order
```

---

48. Thread Safety

A runtime instance is not required to be thread-safe.

The implementation MUST document one of:

```
single-thread confined

externally synchronized

internally synchronized
```

Separate runtime instances SHOULD be independent.

Global mutable interpreter state SHOULD be avoided.

---

49. Determinism

Given:

```
same runtime version

same profile

same canonical modules

same source bytes

same host-provided monotonic context

same resolver profile
```

a deterministic profile MUST produce:

```
same parsed structure
```

```
same evaluated value

same canonical serialization

same validation result

same receipt identity
```

Sources of nondeterminism MUST be explicitly bound into receipt input or prohibited.

Examples:

```
wall clock

random number generator

filesystem iteration order

hash-map iteration order

thread scheduling

floating-point environment
```

---

50. Security Requirements

The user-facing runtime MUST:

```
enforce recursion limits

enforce allocation limits

enforce symbol limits

check integer conversions

reject malformed canonical lengths

reject invalid handles

prevent arbitrary native calls
```

```
separate projection from acceptance

avoid trusting JSON as canonical source

avoid using non-cryptographic hashes as security claims
```

Embedded hosts SHOULD use capability-based native registration.

Untrusted source SHOULD run with:

```
no filesystem

no networking

no process execution

no arbitrary external handles

bounded evaluation fuel
```

---

51. Evaluation Fuel

The runtime SHOULD support a deterministic execution budget:

```c
typedef struct omi_budget {
    uint64_t remaining_steps;
    uint32_t remaining_allocations;
    uint32_t remaining_calls;
} omi_budget;
```

Each evaluation action decrements the budget.

Budget exhaustion returns `OMI_ERR_LIMIT`.

This prevents unbounded recursion and denial-of-service behavior.

---

52. Garbage Collection and Lifetime

A minimal fixed-arena runtime MAY free all objects when the runtime is destroyed.

Hosted implementations MAY provide:

```
mark-and-sweep

reference counting

region reset

generational collection
```

Garbage collection MUST NOT change:

```
value equality

canonical serialization

receipt identity

visible evaluation order
```

---

53. Conformance Test Suite

A conforming runtime MUST test at least the following.

Parsing

```
empty program

atoms

strings

comments

quoted values

proper lists

improper lists

nested dotted pairs
```

```
unexpected right parenthesis

missing right parenthesis

invalid dot position

trailing input
```

Evaluation

```
symbol lookup

unbound symbol error

quote

if true

if false

begin

define

lambda closure

lambda arity error

native arity error

non-callable operator
```

Pair behavior

```
cons

car

cdr

null?

pair?
```

```
proper list detection

improper list rejection where required
```

Numeric primitives

```
rotl16

rotr16

delta16

bqf32

XOR reduction
```

Scope and governor

```
all four scopes

all four subpaths

all five governors

invalid scope

invalid governor

phase reduced modulo 60
```

Quasigroup

```
multiply

left recovery

right recovery

CAR/CDR order sensitivity

order-4 Latin row uniqueness
```

```
order-4 Latin column uniqueness

rank-8 256-bit vectors
```

QuQuart

```
Q0-Q3 values

16 selector masks

no Q4

present-mask behavior
```

Tetrahedral centroid

```
six edge positions

four face positions

evaluated versus passed distinction

centroid reduction

missing-coordinate recovery
```

Serialization

```
same value gives same bytes

reversed pair gives different bytes

proper and improper lists differ

hash-map order does not affect bytes

cycles rejected in tree profile
```

Receipt

```
slot range 0..5039

same route gives same receipt

changed source changes receipt

changed notation changes receipt

changed reading changes receipt

changed result changes receipt

changed resolver profile changes receipt

validation rejection remains recorded as rejection
```

---

54. Required Reference Vectors

At minimum:

```
(NULL . NULL)
```

Expected:

```
ordered pair

contrast witness may be 0

structural digest nonempty

not confused with bare NIL
```

```
(0xAA55 . 0x55AA)
```

Expected:

```
contrast witness = 0xFFFF
```

Reverse:

```
(0x55AA . 0xAA55)
```

Expected:

```
same contrast witness allowed

different structural digest required
```

Nested scopes:

```
(FS . (GS . (RS . US)))
```

Expected:

```
ordered nested pair structure preserved
```

QuQuart:

```
(ququart source notation reading receipt)
```

Expected:

```
four states only
```

Centroid:

```
(centroid-reduce RULES FACTS COMBINATORS CLOSURES)
```

Expected:

```
CONS result
edge masks
```

```
face masks
validation metadata
```

---

55. Python Reference Compatibility

The C implementation SHOULD initially match the Python reference for:

```
tokenization

parsing

Pair formatting

proper/improper-list behavior

truthiness

special-form behavior

scope masks

subpath masks

governor exponents

Delta16

BQF

NULL-ring XOR

Polybius diagonal XOR

default Omnicron envelope

receipt-slot calculation
```

Where this specification intentionally strengthens behavior, the C implementation MUST follow this specification and the Python reference SHOULD later be updated.

Notable strengthened areas:

```
ordered canonical CONS identity

explicit quasigroup profiles

evaluated/passed masks

capability isolation

deterministic receipt inputs

bounded runtime limits
```

---

56. User-Facing Stability Commitments

The runtime project SHOULD publish stability levels:

```
Language syntax
stable after 1.0

C ABI
versioned and backward-compatible within major version

canonical serialization
immutable per profile version

receipt format
versioned

quasigroup profiles
identified by stable numeric IDs

module contracts
versioned independently

CLI text output
human-facing and not canonical

JSON
projection format unless typed profile says otherwise
```

---

57. Versioning

Recommended version fields:

```
runtime semantic version

C ABI version

language grammar version

canonical serialization version

receipt profile version

quasigroup profile version

validation profile version

module schema version
```

A receipt MUST bind all profile identifiers that affect its result.

---

58. Implementation Phases

Phase 1 — Semantic Core

Implement:

```
value handles

arena

symbols

lexer

parser

Pair

formatting
```

```
environment

evaluator

special forms

native registry
```

Phase 2 — Deterministic OMI Primitives

Implement:

```
scope

governors

rotations

Delta16

BQF

canonical serialization
```

Phase 3 — Quasigroup and QuQuart

Implement:

```
quasigroup ABI

order-4 reference profile

rank-8 word256 profile

QuQuart

tetrahedral edges

tetrahedral faces

centroid reducer
```

Phase 4 — Validation and Receipt

Implement:

```
validation profiles

receipt construction

receipt replay stability

5040 ring

host persistence
```

Phase 5 — Embedding

Implement:

```
stable shared library

Python binding

Node/WASM binding

Rust wrapper

Go wrapper
```

Phase 6 — User-Facing Modules

Implement:

```
RULES.o

FACTS.o

COMBINATORS.o

CLOSURES.o

CONS.o

module loader
```

```
profile manifests
```

---

59. Minimum Viable Public Release

A first public release SHOULD include:

```
libomi shared and static libraries

public headers

omi CLI

interactive REPL

minimal hosted profile

minimal embedded profile

canonical serialization specification

order-4 quasigroup profile

rank-8 predicate type

QuQuart and centroid APIs

receipt replay stability

C tests

Python equivalence tests

WASM smoke test

user guide

embedding guide

security guide
```

---

60. Canonical User-Facing Examples

Basic Pair

```
(cons 'RULES 'FACTS)
```

Dotted Pair

```
(RULES . FACTS)
```

Grounded Source

```
(define source (cons RULES FACTS))
```

Candidate Reading

```
(define reading
  (COMBINATORS source notation))
```

Closure

```
(define receipt-candidate
  (CLOSURES source notation reading))
```

Centroid

```
(CONS source notation reading receipt-candidate)
```

Explicit Quasigroup Resolution

```
(quasigroup-resolve profile car cdr)
```

Recovery

```
(quasigroup-left-divide profile car cons-result)

(quasigroup-right-divide profile cons-result cdr)
```

Receipt

```
(cite
  (CONS source notation reading receipt-candidate))
```

---

61. Canonical End-to-End Execution

```
1. Create runtime.

2. Select a named profile.

3. Register host capabilities.

4. Load RULES.o.

5. Load FACTS.o.

6. Load COMBINATORS.o.

7. Load CLOSURES.o.

8. Load CONS.o.

9. Stage Omicron readiness if strict mode is enabled.

10. Parse source.

11. Build Q0 from RULES and FACTS.

12. Produce Q1 notation.

13. Select CDR continuation or quasigroup profile.

14. Apply COMBINATORS to produce Q2.
```

```
15. Evaluate six tetrahedral edges.

16. Evaluate four tetrahedral faces.

17. Apply CLOSURES to produce a Q3 candidate.

18. Apply CONS centroid reduction.

19. Recover missing bounded coordinates if requested.

20. Run validation.

21. Canonically serialize the accepted or rejected result.

22. Construct receipt.

23. Place receipt in the configured ring or host storage.

24. Replay the same declared route.

25. Confirm receipt identity.

26. Return the user-facing result and receipt projection.
```

---

62. Final Authority Boundaries

```
Parsing recognizes syntax.

Evaluation produces values.

Quasigroup resolution recovers coordinates.

QuQuart organizes interpretation.

CONS reduces the tetrahedral cell.

Tetragrammatron may govern larger closure.

Metatron may record incidence.

Projection exposes a result.

Validation decides acceptance.
```

```
Receipt records and stabilizes that decision.
```

No earlier layer may impersonate a later authority.

---

63. Final Canon Lock

```
OMI-Lisp is a portable deterministic interpretation language.

The Python implementation is the reference oracle.

The C implementation is the portable semantic runtime.
```

```
The runtime exposes four QuQuart states:

Q0 source
Q1 notation
Q2 reading
Q3 receipt.
```

```
RULES.o supplies source law.

FACTS.o grounds the source.

COMBINATORS.o supplies reading operations.

CLOSURES.o supplies closure and receipt boundaries.

CONS.o supplies centroid reduction and reversible continuation.
```

```
CONS is not Q4.

CONS binds Q0-Q3 as the centroid relation.
```

```
CAR is ordered before CDR.

CAR and CDR must not be collapsed
into a commutative structural identity.
```

```
The Boolean ladder defines capacity.

At rank n:

CAR has 2^n assignments.

One CDR occupies 2^n bits.

There are 2^(2^n) possible CDR functions.
```

```
The quasigroup defines reversible resolution:

CAR × CDR -> CONS

CAR × CONS -> CDR

CONS × CDR -> CAR.
```

```
The Latin-square law may be implemented algorithmically.

A giant table is not required.
```

```
The C ABI uses opaque handles.

The core uses bounded memory.

The host supplies explicit capabilities.

Every serialized identity is ordered, typed, and versioned.
```

```
The same runtime may be embedded in:

C
C++
Python
JavaScript
Rust
Go
Java
WebAssembly
firmware
or another process.
```

```
Same source,
same notation,
same route,
same resolver,
same result,
same receipt context

must produce the same receipt identity.
```

```
Validation accepts.

Receipt records.

Replay confirms stability.
```